
Automatic File Extraction Example

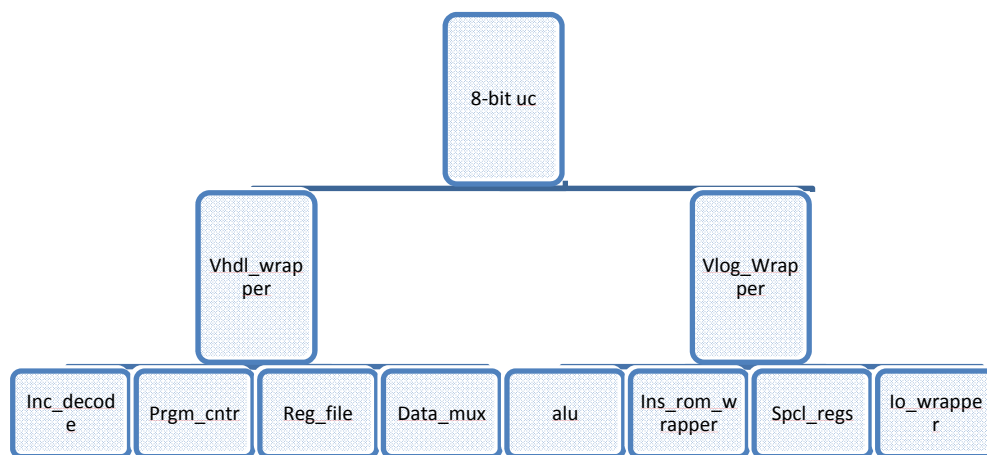
Purpose

This example demonstrates how the Automatic File Extraction features can help the designer quickly create a sub-project.

Demo Design

The example design is an 8-bit microprocessor, which is implemented in a Mixed HDL having 2 chains each for VHDL and Verilog. Each chain will demonstrate the different feature aspects for the corresponding language.

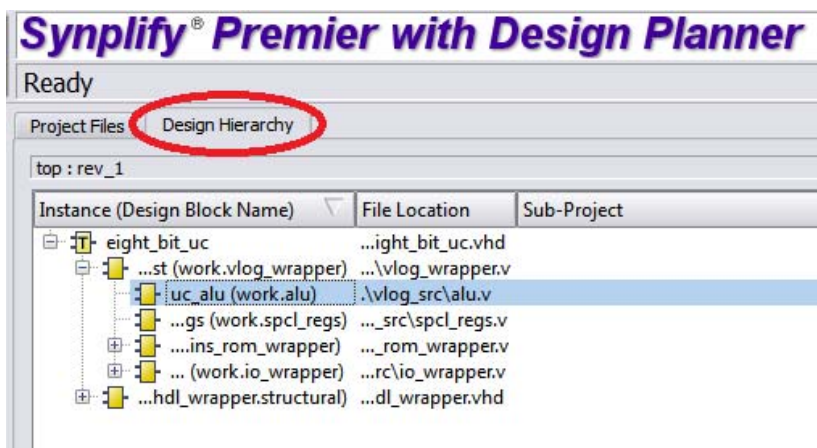
The hierarchy is as follows:



Steps for Demo

Compile Top-level Design

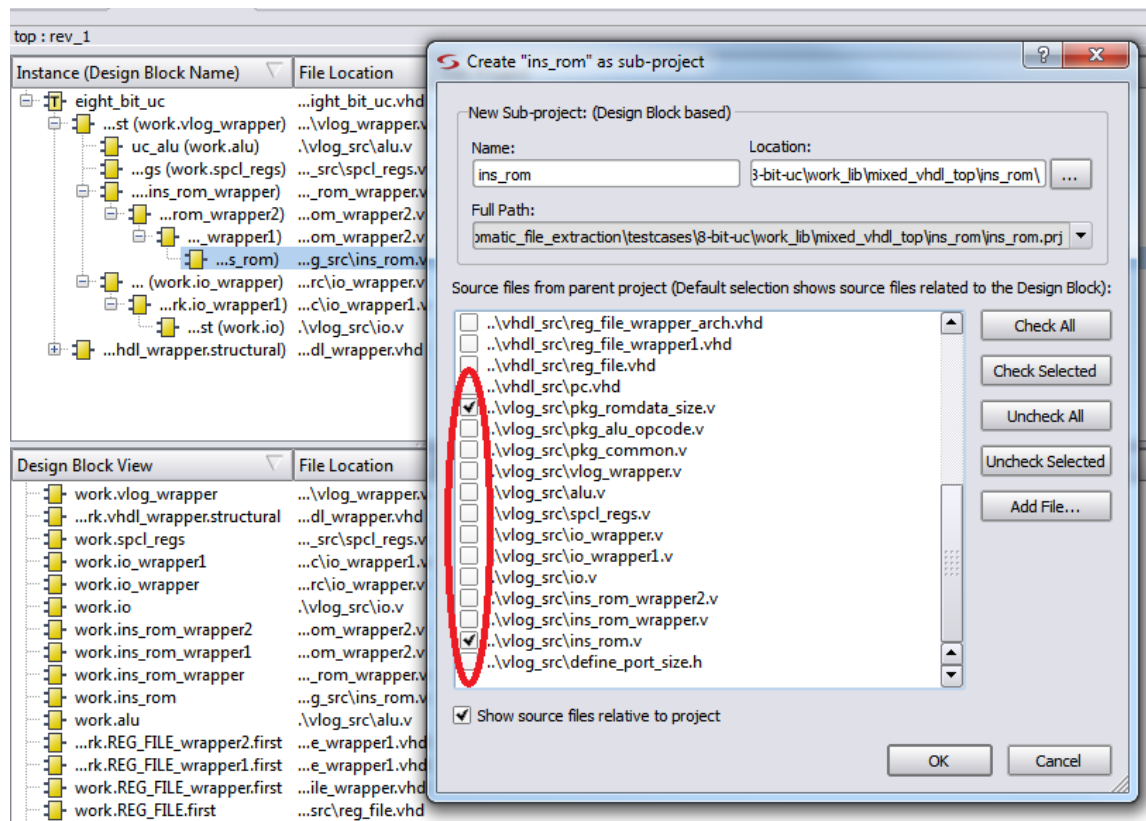
Once the Design is compiled, “Design Hierarchy” view displays the details of the design.



Create Sub-projects for Verilog Modules

Create sub-project for “ins_rom”:

- In “Design Hierarchy” view right-click ins_rom instance
- Select “Create Sub-project (Design Block)” from the popup menu.
- All the files required for Design Block will be automatically selected
- Click OK

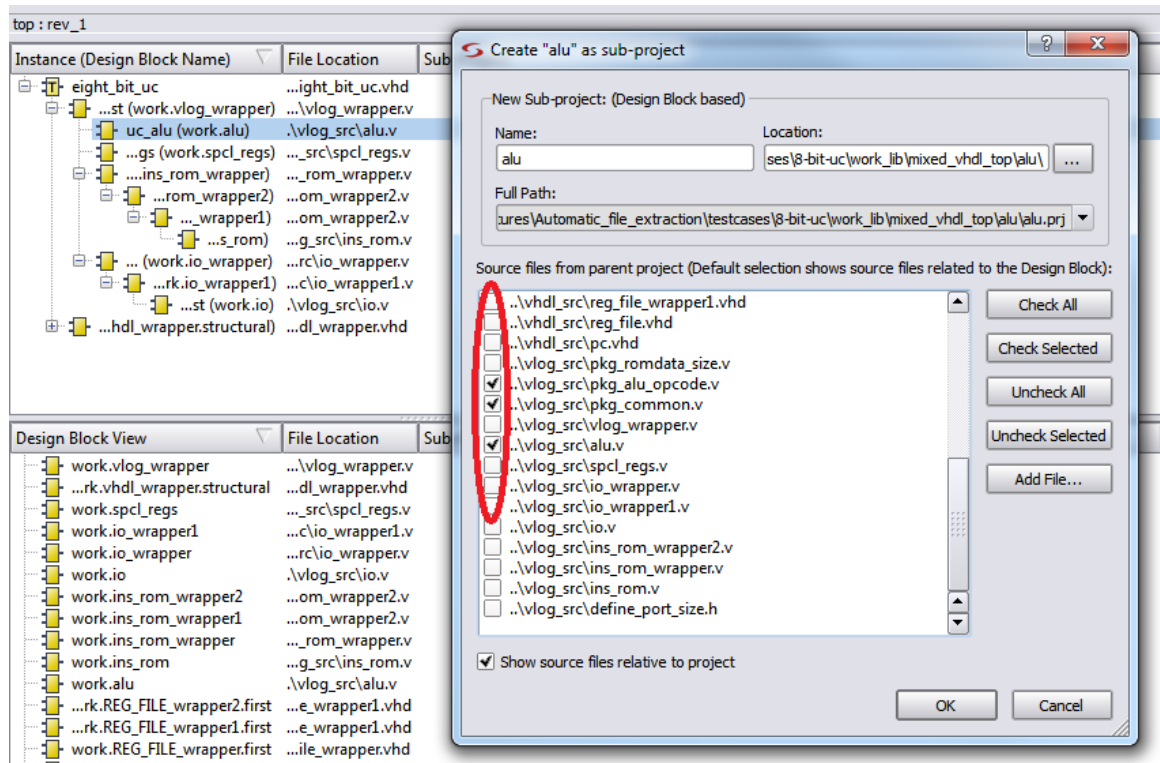


Observation:

“ins_rom” uses parameter from the “pkg_romdata_size” from “pkg_romdata_size”. Hence, “ins_rom.v” and “pkg_romdata_size.v” files are autoselected.

Create sub-project for “alu”:

- In “Design Hierarchy” view right-click “alu” instance
- Select “Create Sub-project (Design Block)” from the popup menu.
- All the files required for Design Block will be automatically selected
- Click OK

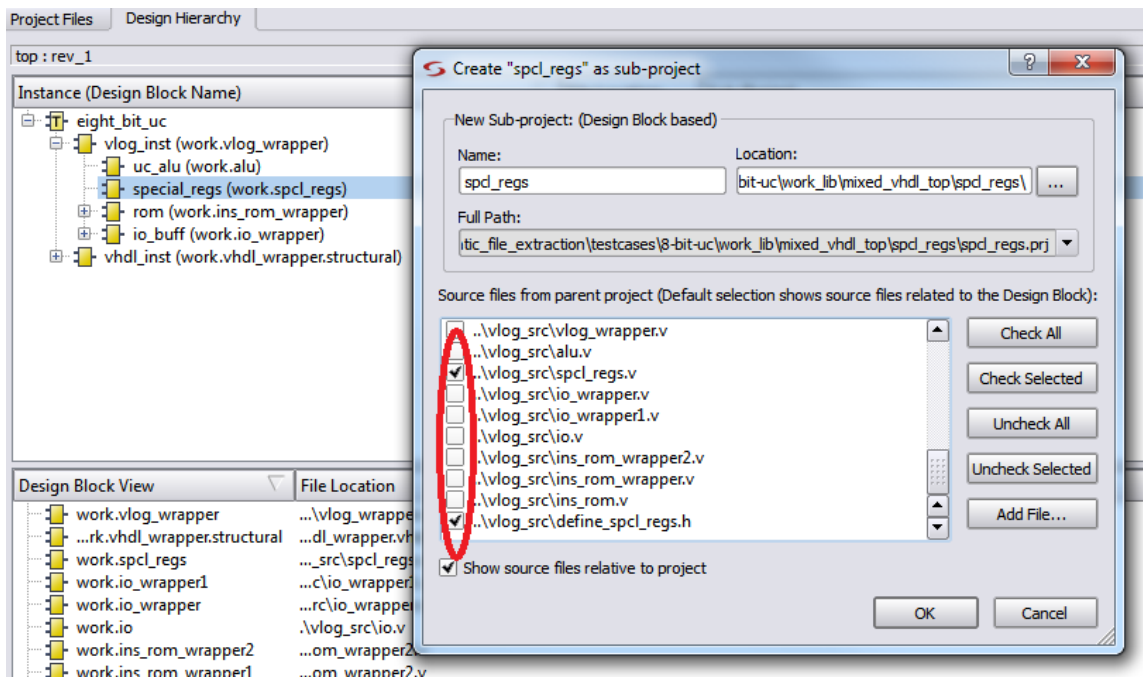


Observation:

“alu” uses parameters from the package “pkg_common”. “pkg_common” internally imports elements from “pkg_alu_opcode”. And hence, “pkg_alu_opcode.v”, “pkg_common.v” and “alu.v” are auto-selected.

Create sub-project for “spcl_regs”:

- In “Design Hierarchy” view right-click “spcl_regs” instance
- Select “Create Sub-project (Design Block)” from the popup menu.
- All the files required for Design Block will be automatically selected
- Click OK

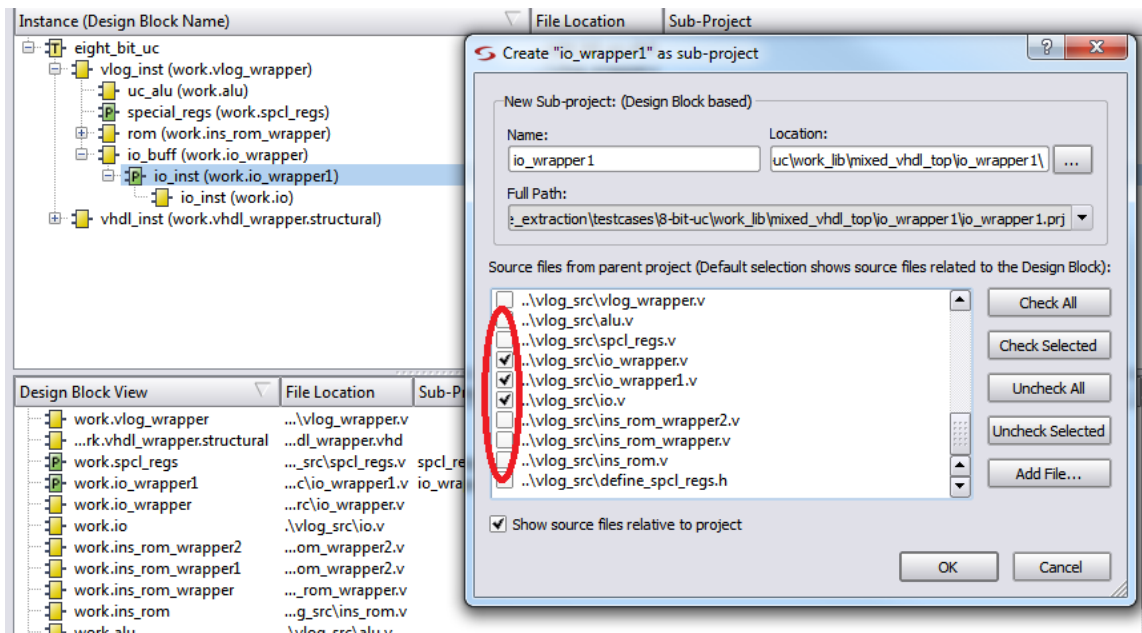


Observation:

“spcl_regs” uses macro `alu_size` from the header file “define_spcl_regs.h” and this header file is added in the main project. Hence “define_spcl_regs.” and “spcl_regs.v” are auto-selected.

Create sub-project for “io_wrapper1”:

- In “Design Hierarchy” view right-click “io_wrapper1” instance
- Select “Create Sub-project (Design Block)” from the popup menu.
- All the files required for Design Block will be automatically selected
- Click OK



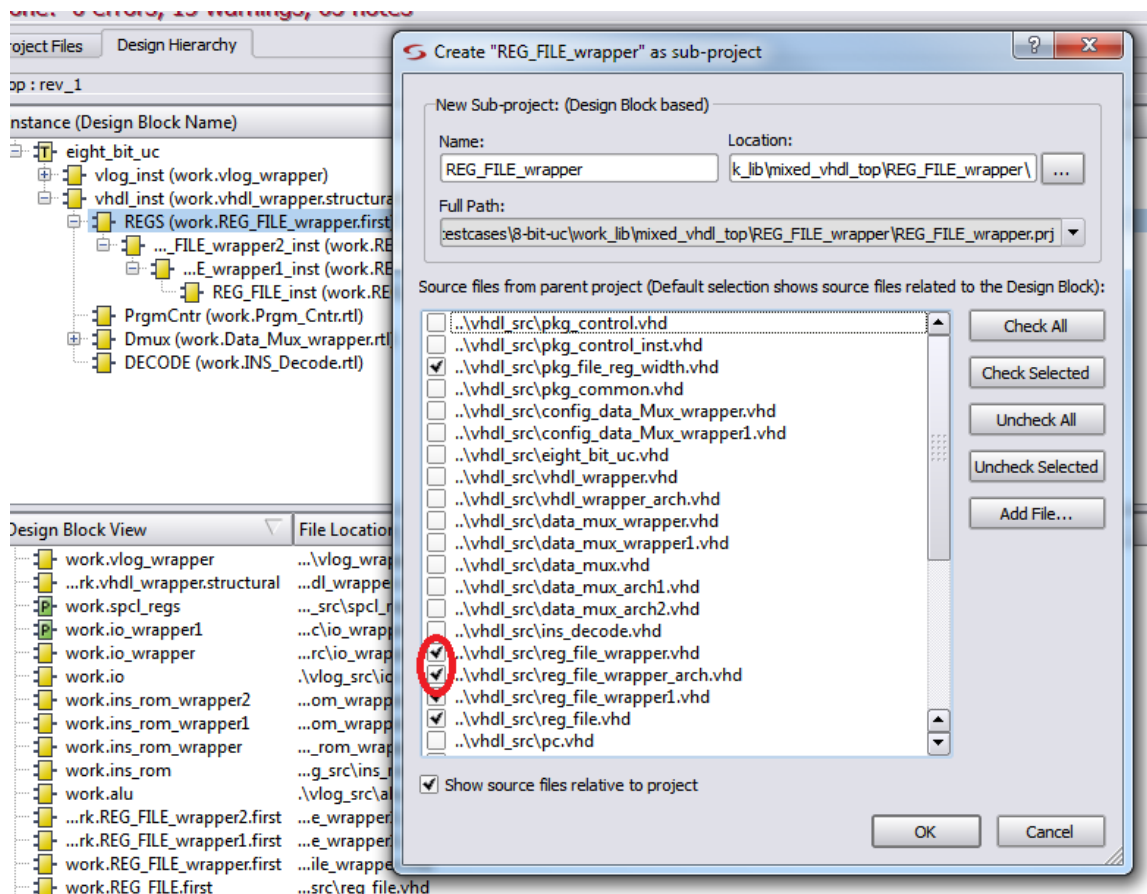
Observation:

“io_wrapper1” uses **macro** SOC_TYPE from the header file “define_port_size.h” which is included in the parent file “io_wrapper.v”. Also module “io” is instantiated in “io_wrapper1.v”. Hence “io_wrapper1.v”, “io_wrapper.v” and “io.v” are auto-selected.

Create sub projects in VHDL Modules

Create sub-project for “reg_file_wrapper”:

- In “Design Hierarchy” view right-click “reg_file_wrapper” instance
- Select “Create Sub-project (Design Block)” from the popup menu.
- All the files required for Design Block will be automatically selected
- Click OK

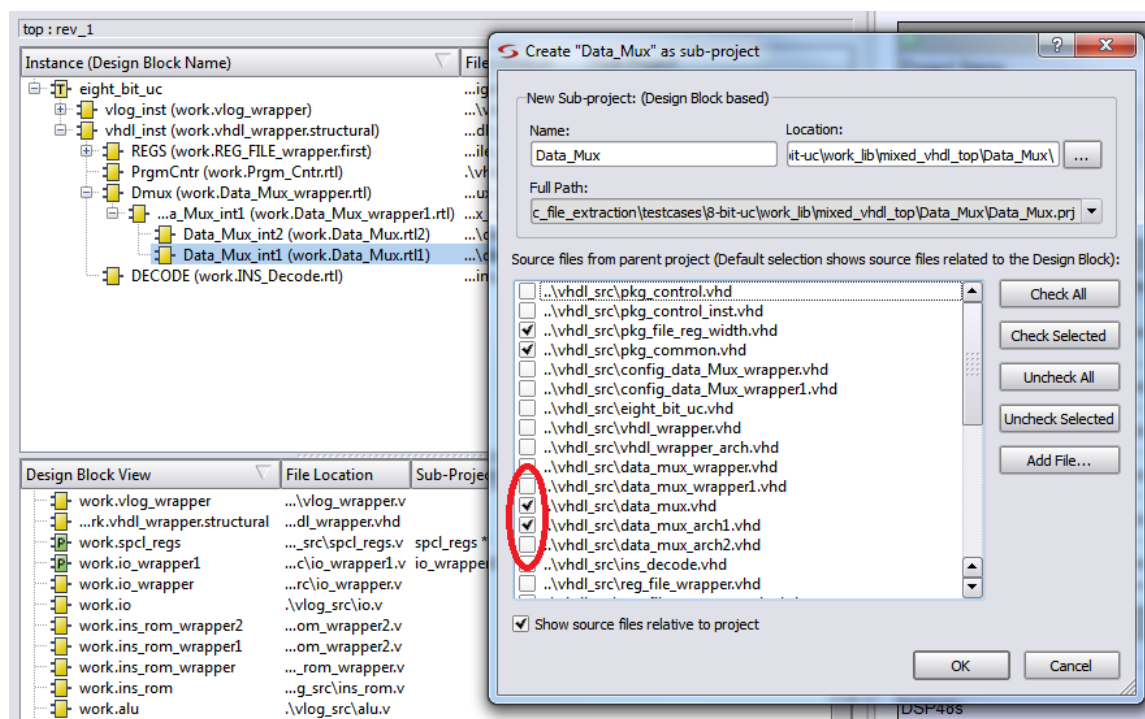


Observation:

“reg_file_wrapper” uses architecture which is defined in the file “reg_file_wrapper_arch.vhd”. Also module “reg_file_wrapper1” is instantiated in “reg_file_wrapper.vhd”. Hence “reg_file_wrapper.vhd” “reg_file_wrapper_arch.vhd” and “reg_file_wrapper1.vhd” are auto-selected.

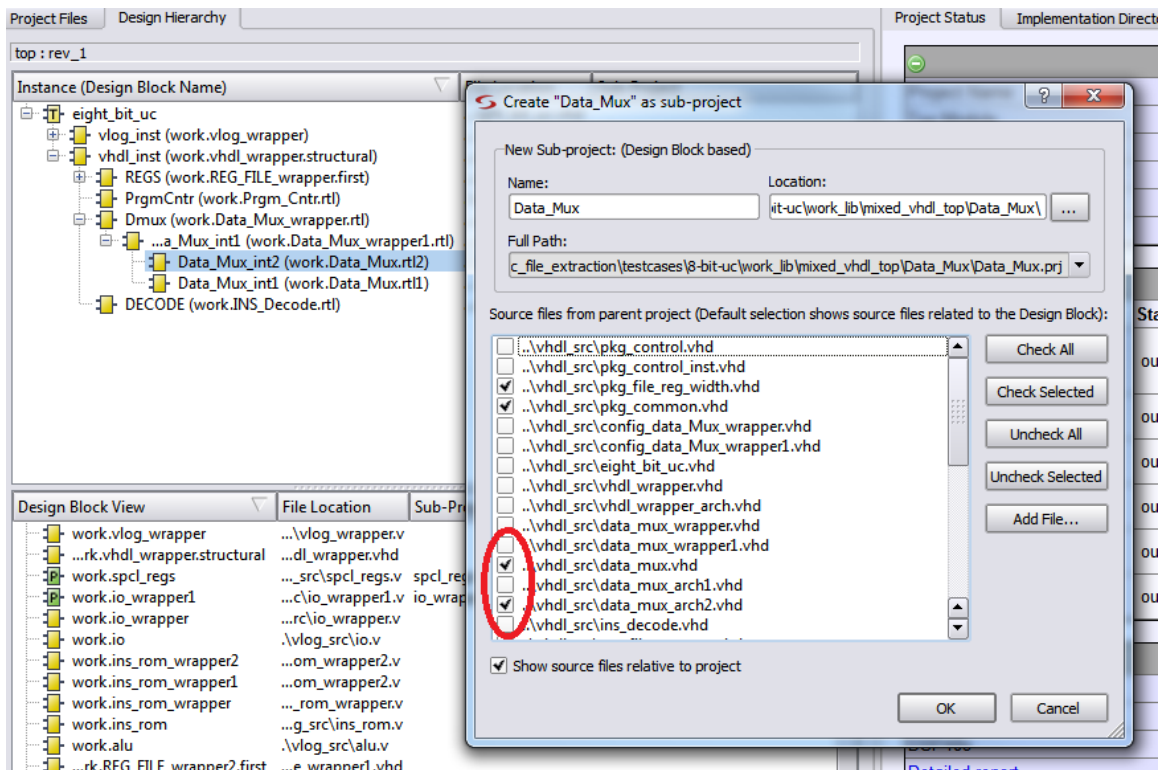
Create sub-project for “data mux”:

- In “Design Hierarchy” view right-click “data_mux” instance
- Select “Create Sub-project (Design Block)” from the popup menu.
- All the files required for Design Block will be automatically selected
- Click OK



Observation (if *work.Data_Mux.rtl1* is selected):

“data_mux_wrapper1” instantiates 2 instances of “data_mux” and is configured by configuration file “config_data_mux_wrapper1.vhd”. The configuration selects architecture RTL1 for instance “Data_Mux_int1” and RTL2 for instance when Data_Mux_int2. Hence when sub project is created using instance “data_mux” the corresponding architecture “data_mux_wrapper1_arch.vhd” is auto-selected.

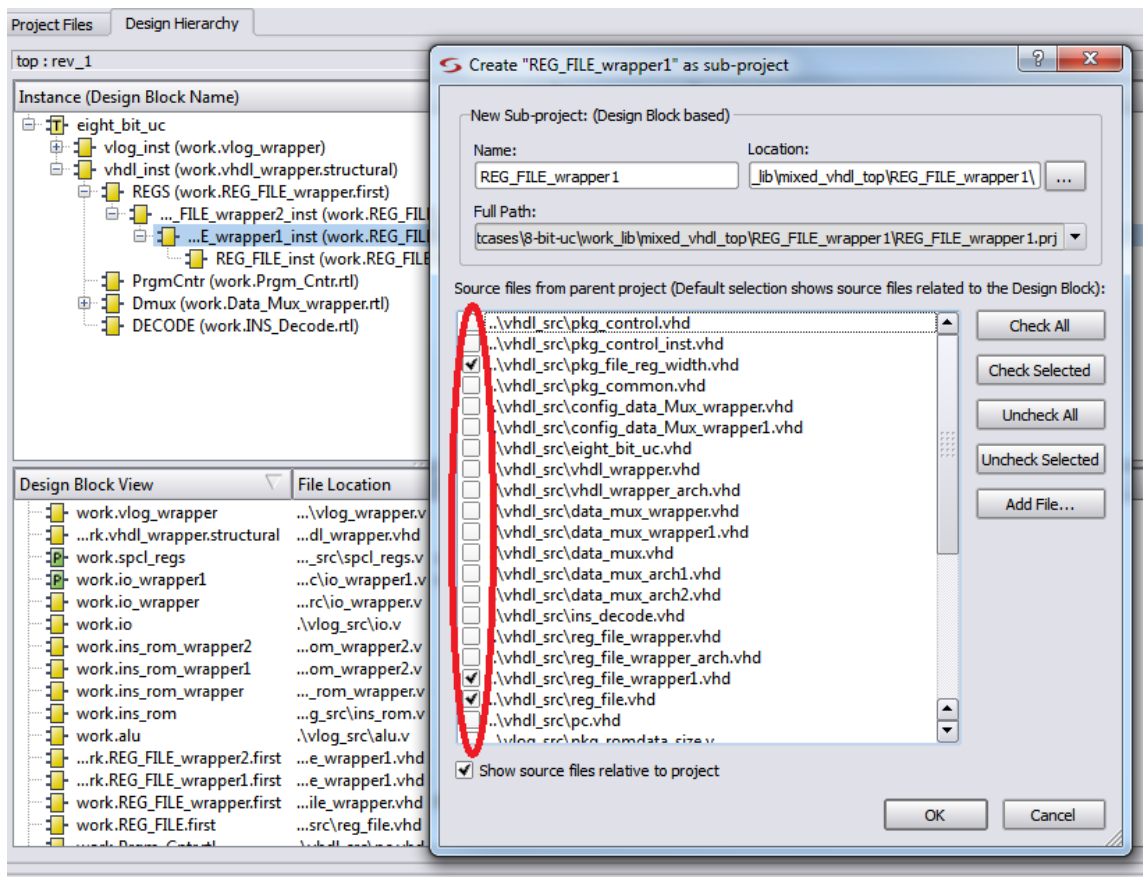


Observation (if `work.Data_Mux.rtl2` is selected):

“data_mux_wrapper1” instantiates 2 instances of “data_mux” and is configured by configuration file “config_data_mux_wrapper1.vhd”. The configuration selects architecture RTL1 for instance “Data_Mux_int1” and RTL2 for instance when Data_Mux_int2. Hence when sub project is created using instance “data_mux” the corresponding architecture “data_mux_wrapper2_arch.vhd” is auto-selected.

Create sub-project for “reg_file_wrapper1”:

- In “Design Hierarchy” view right-click “reg_file_wrapper1” instance
- Select “Create Sub-project (Design Block)” from the popup menu.
- All the files required for Design Block will be automatically selected
- Click OK

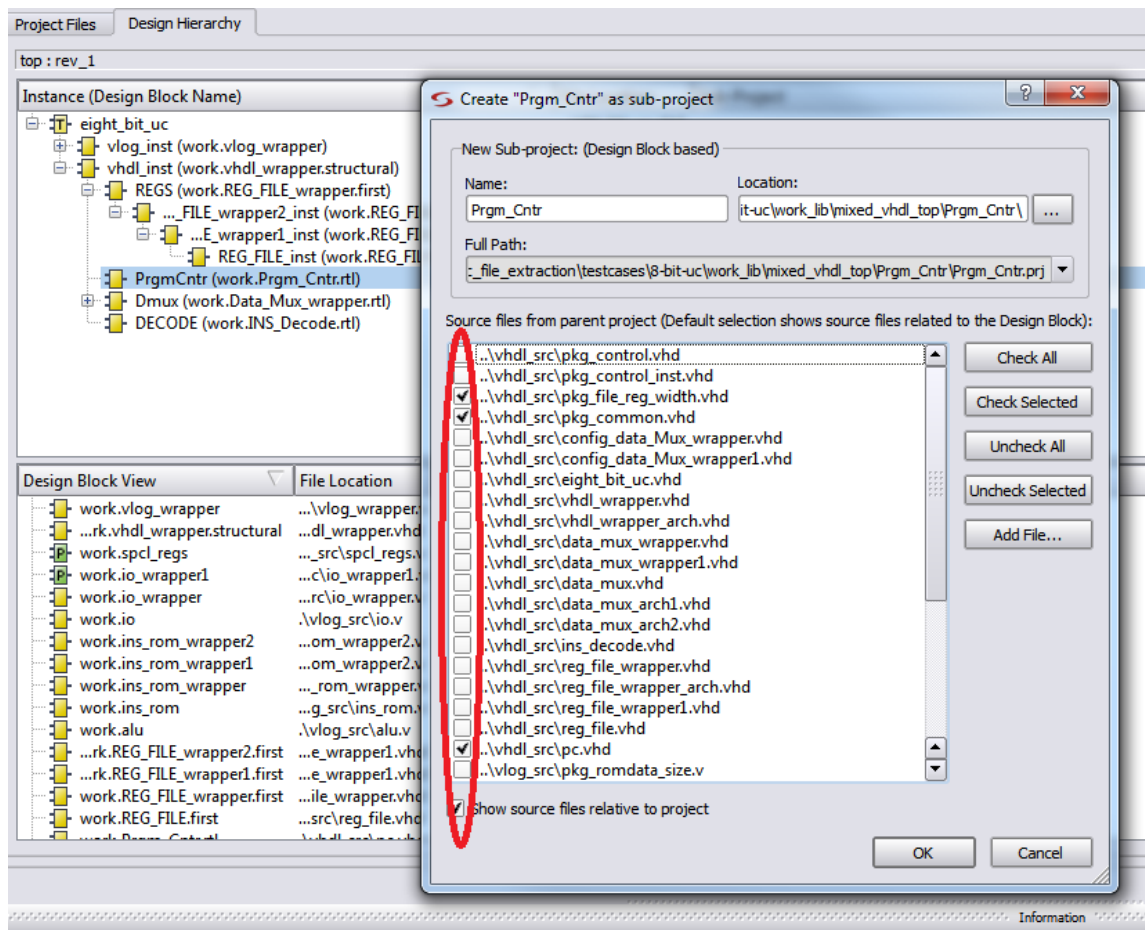


Observation:

“reg_file_wrapper1” uses parameters from the “pkg_file_reg_width”. Hence, “reg_file_wrapper1.vhd” and “pkg_file_reg_width.vhd” files are autoselected.

Create sub-project for “pc”:

- In “Design Hierarchy” view right-click “pc” instance
- Select “Create Sub-project (Design Block)” from the popup menu.
- All the files required for Design Block will be automatically selected
- Click OK

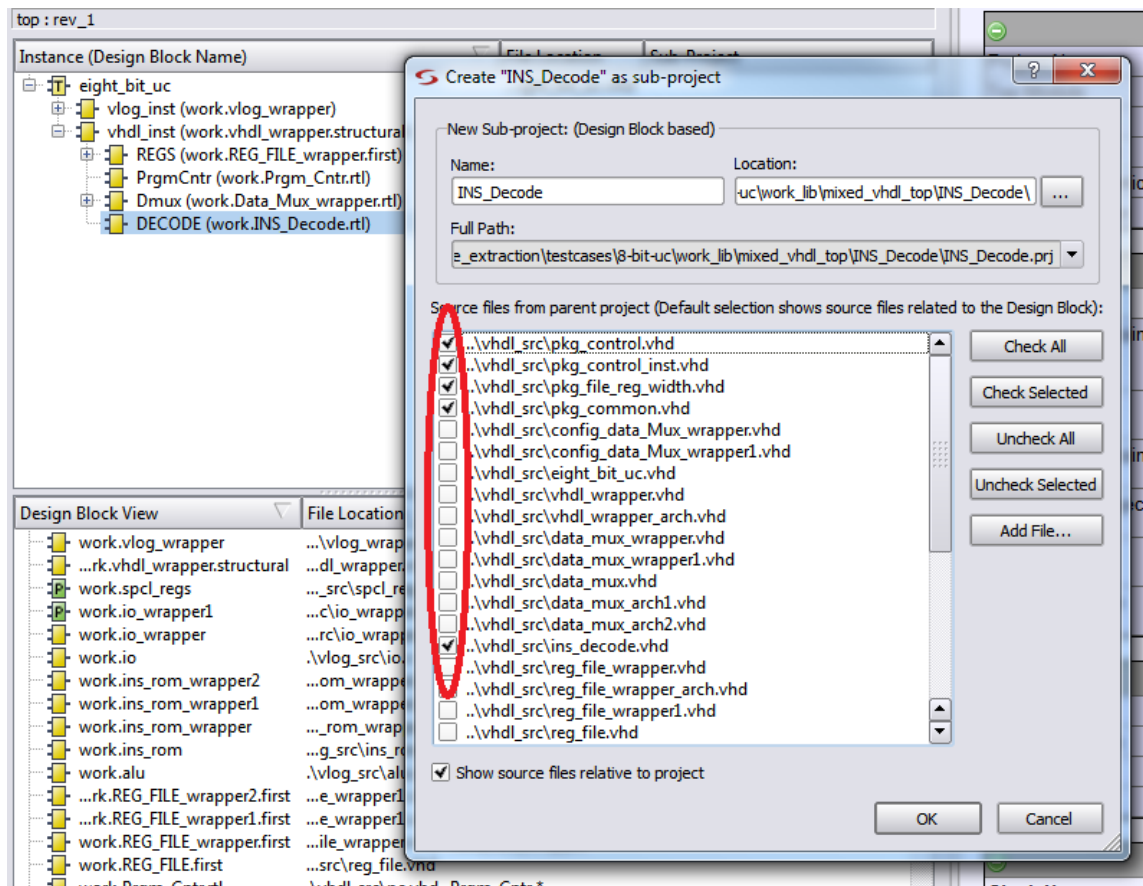


Observation:

“pc” uses parameters from the package “pkg_common”. “pkg_common” internally imports elements from “pkg_file_reg_width”. And hence, “pkg_file_reg_width.vhd”, “pkg_common.vhd” and “pc.vhd” are auto-selected.

Create sub-project for “ins_decode”:

- In “Design Hierarchy” view right-click “ins_decode” instance
- Select “Create Sub-project (Design Block)” from the popup menu.
- All the files required for Design Block will be automatically selected
- Click OK



Observation:

“ins_decode” uses parameters from the package “pkg_control_inst”. “pkg_control_inst” internally **instantiates another package** pkg_control.vhd . Also it uses parameters from the package “pkg_common”. “pkg_common” internally imports elements from “pkg_file_reg_width”. And hence, “pkg_control_inst” , “pkg_control” “pkg_file_reg_width.vhd”, “pkg_common.vhd” and “ins_decode.vhd” are auto-selected.

Feature Summary

Verilog

- Package files
 - Packages can be imported in different ways like Import through wild card * or Import necessary variables or Direct import through :: . In all the scenarios, the package files are automatically extracted when sub projects are created.
- Nested packages files
 - In scenarios where nested package is used, the necessary package files are added when sub projects are created.
- Include files
 - Includes files if added to the main project gets added when sub projects are created using the module in which the include files are included.
- Macros
 - In scenarios where Macros are defined in Parent module and used in sub modules, Parent module are automatically added when sub project is created using sub modules.

VHDL

- Entity and architecture in separate file
 - In scenarios where entity and architecture are in separate files, entity file is linked to the correct architecture file when sub project is created.
- Multiple architectures in separate files
 - In scenarios where entity is having multiple architectures in different files, entity file is linked to the correct architecture file when sub project is created.
- Configuration files
 - When configuration files are used, the necessary configuration details are passed and when sub projects are created.
- Package files
 - In scenarios where package files are used the required package files are automatically extracted when sub projects are created
- Nested Package files
 - In scenarios where nested package is used , the necessary package files are added when sub projects are created.
- VHDL 2008
 - In scenarios where vhd12008 features like Package instantiations are used the required files are automatically loaded.

Limitations

1. In Mixed HDL, the feature is limited to language boundaries while automatically identifying the dependent files
2. Verilog XMR is not supported. If the design uses XMR, then the dependent files related to XMR are not automatically identified.
3. Verilog Configuration is not supported.
4. System Verilog Interface is not supported. If the design uses SystemVerilog Interface, then the dependent files related to Interface are not automatically identified.